



Xi'an Jiaotong-Liverpool University

西交利物浦大學

MFE113TC

Programming Foundation for Intelligent Manufacturing Engineering

Lecture 05 Functions in C/C++

Dr. Jiachen Liu
jiachen.liu@xjtlu.edu.cn

Fall, 2026

Outline

- Function Intros;
- Commonly-used Library;
- User-defined Functions;
- Variable Scope;
- Function Calling;
- C Preprocessor;
- Random Number Generation;



Introduction

- Computer programs that solve real-world programs are usually made of smaller and manageable pieces of code or **modules**;
- In C programming, modules are called **functions**;

Introduction

What is a function?

- A function is a block of code that performs a **specific task**;
- A function is able to:
 - perform an action: e.g., `printf()` display data;
 - provide a value: e.g., `sqrt()` computes the square root of a positive real number (in **`<math.h>`**).

Introduction

Why we want to use functions?

- It enables high level abstraction;
- It avoids repetitious programming;
- It makes your codes **modular**:
 - Easy to read;
 - Easy to maintain;
 - Easy to test and fix.

Introduction

Why we want to use functions?

- Which one looks easier to read? How about a more complicated program?

```
int main()
{
    printf("Hello\n");
    char name[] = "James";
    printf("My name is %s\n",name);
    int age = 19;
    printf("I'm %d years old\n",age);
    printf("I'm a rookie in programming.\n");
    printf("I can draw a simple map in C.\n");

    printf("*****\n");
    printf("*S                *\n");
    printf("* ***** *\n");
    printf("* ***** *\n");
    printf("*                E*\n");
    printf("*****\n");
    return 0;
}
```

```
int main()
{
    self_introduce();
    draw_map();
    return 0;
}
```

Introduction

Why we want to use functions?

- Which one looks easier to read? How about a more complicated program?

```
int main()
{
    printf("Hello\n");
    char name[] = "James";
    printf("My name is %s\n",name);
    int age = 19;
    printf("I'm %d years old\n",age);
    printf("I'm a rookie in programming.\n");
    printf("I can draw a simple map in C.\n");

    printf("*****\n");
    printf("S          *\n");
    printf("* ***** *\n");
    printf("* ***** *\n");
    printf("*          E*\n");
    printf("*****\n");
    return 0;
}
```

```
int main()
{
    self_introduce();
    draw_map();
    return 0;
}
```

Introduction

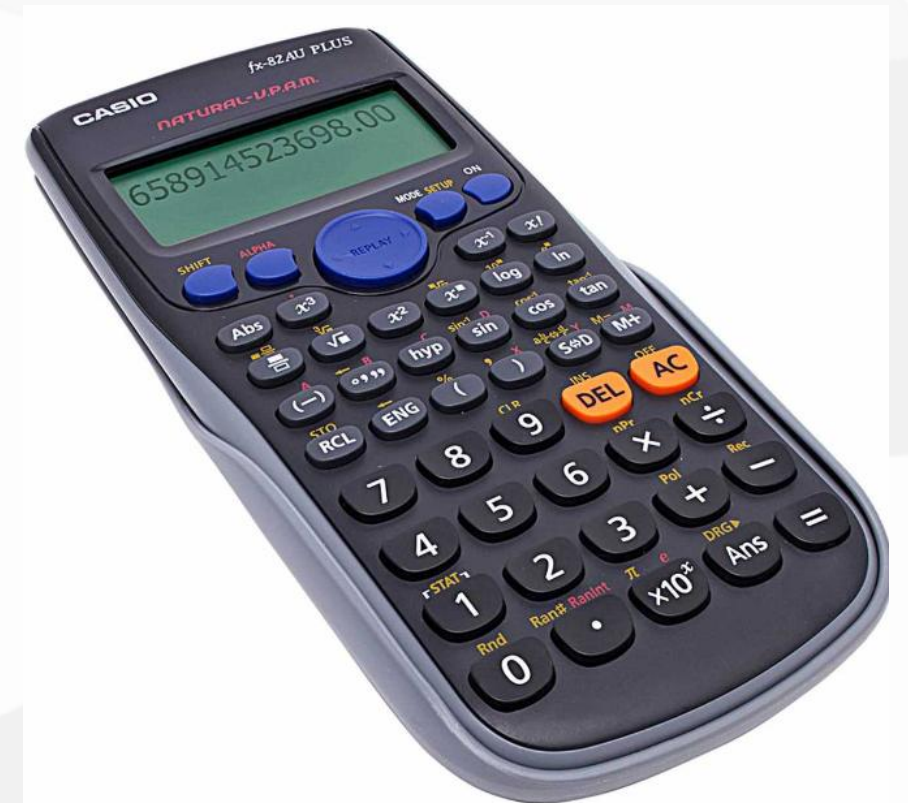
Why we want to use functions?

- **High-level abstraction** allows us to use various functions easily, even without understanding the intricate details of how they work.
- We have already used a lot of pre-defined functions. Like `printf()`, `scanf()`, `std::cin`, `std::cout`.
- `printf()` and `scanf()` are functions defined in `<stdio.h>`. However, we aren't required to understand their inner workings.

Commonly-used Library

- The header file `<math.h>` provides useful functions for fundamental mathematical operations.

Function	Description
<code>acos(x)</code>	returns the value of <code>arccos(x)</code> .
<code>atan(x)</code>	returns the value of <code>arctan(x)</code> .
<code>exp(x)</code>	returns the value of e^x
<code>pow(x,y)</code>	returns the value of x^y
<code>sqrt(x)</code>	returns the value of $\sqrt{x}$
etc.	etc.



Commonly-used Library

- The header file `<ctype.h>` contains useful functions to perform tests and manipulations of character data.

Function	Description
<code>isdigit(x)</code>	returns 1 if x is a digit (0-9), else 0.
<code>isalpha(x)</code>	returns 1 if x is a letter (Aa-Zz), else 0.
<code>isalnum(x)</code>	returns 1 if x is a digit or letter, else 0.
<code>islower(x)</code>	returns 1 if x is a lowercase letter (a-z), else 0.
<code>toupper(x)</code>	converts x to a lowercase letter if it is uppercase.
etc.	etc.

User-defined Functions

- So far, we have written programs consisting of a function called **main**;
- Programmers can write their own customized functions;
- Each function should be limited to performing **a single, well-defined task**;
- The function **name** should **reflects that task**. (Remember? In **Lecture 02**, we learned what a descriptive name is.)



User-defined Functions

A Modular Program

```
#include <stdio.h>
int square (int);           /*function prototype*/
int main (void){
    int x;
    for (x=1, x<=10; x++)
        {printf("%d ", square(x));
          printf ("\n");}
    return 0;
}                             /*End of main()*/

/* Function definition */
int square (int y) {
    return y*y;
}
```

User-defined Functions

A Modular Program

```
1. #include <stdio.h>
   int square (int);           /*function prototype*/
   int main (void){
       int x;
       for (x=1, x<=10; x++)
           {printf("%d ", square(x));
            printf ("\n");}
       return 0;
   }                               /*End of main()*/

   /* Function definition */
2. int square (int y) {
       return y*y;
   }
```

User-defined Functions

Function recipe

- Step 1: **Declare the function (similar as declaring a variable):**

e.g., `int square(int);`

It determines what kind of value (if any) that the function will return (only one return value)

It determines the type of values that the function will receive (can be more than one), in this case, it is an integer value that is sent to the function.

User-defined Functions

Function recipe

- Step 1: **Declare the function (similar as declaring a variable):**

e.g., `int square(int);`

- Step 2: **Define the function:**

e.g., `int square(int y){ /* declare the variable y for the input argument */
 return y*y; /* write some code for the implementation */
} /* perform operations and return */`

- Step 3: **Call / Use the function:**

e.g., `square(x)`

`square(1)`

`square(3+5);`

User-defined Functions

Function Classification

- In C/C++, functions can be classified into two types based on whether they need to return a value or not. If they do not:

```
void printHello();           // Prototype
```

```
void printHello(){ // definition  
    printf("hello, world.");  
}
```

 No return value.

User-defined Functions

Function Classification

- Return values in C/C++ can be of various types, such as `int`, `float`, `double`, and `char`. However, a function cannot return an `array` directly.

```
float square(float);           // Prototype
float square(float x){         // definition
    float res = x*x;
    return res;
}
```

- Recall from [Lecture 02](#) that the `return` statement not only provides a value but also signifies **the termination of a function**.

User-defined Functions

Function Declarations

- A function without any arguments or return value:

void function_name();

- A function with arguments and no return value:

void function_name(**int**, **float**, **char**, ..., **int**);

- A function with no arguments but a return value:

float function_name();

- A function with arguments and a return value:

float function_name(**float**, **char**, ..., **int**);

User-defined Functions

Function Arguments

```
int multiply (int z, int y){  
    int x;  
    z = z + 1;  
    x = z * y;  
    return x;  
}
```

- int z and int y are formal arguments or parameters (inputs)
- z, x and y are **private** to the function (only valid inside the function);
- The parameters z and y receive values when the function is called.

```
int z = 5, x = 10, y;  
y = multiply(z,25);
```

- 5 and 25 are the actual arguments sent to the function.

User-defined Functions

Function Arguments

- As well as constant values and variables, functions (with **return** values) may be placed in the function call as an argument:

```
x = multiply(add(x));
```

- You can also perform operations in the function call:

```
x=multiply(x+y);
```

- Whatever operations you place in the function call, be careful about that the result must be unaffected by the processing order, as it is not defined in C programming.

```
x = multiply(add(x)-add(y));
```

If we are not familiar with the definition of add(). We should not use it this way.

User-defined Functions

The order of the argument evaluation

- If some of the arguments are expressions or function calls, we have to make sure that the result is **irrelevant to the order of the corresponding evaluation**. For example:

```
x = multiply(add(x)-add(y));
```

- Therefore, you need to use temporary variable.

```
a = add(x);
```

```
b = add(y);
```

```
x = multiply(a-b);
```

- This approach ensures that `add(x)` is evaluated before `add(y)`, thus avoiding any ambiguity in the order of evaluation.

User-defined Functions

Function **return** Value

- Functions can **return** a value (output);
- The **return** keyword indicates what will be returned at the end of the function implementation:
 - The returned value should be the **same type** as the function type;
 - Otherwise, **automatic casting (implicit data type conversion)** will occur.
- When **return** is reached, the function will be terminated.

```
int multiply (int x, int y)
{
    int z;
    z = x*y;
    return z;
}
```

User-defined Functions

Function **return** Value

- The returned value should be the **same type** as the function type;
- Otherwise, **automatic casting (implicit data type conversion)** will occur.

```
#include <stdio.h>
int cast_test();
int main()
{
    int a = cast_test();
    printf("%d\n", a);
    return 0;
}

int cast_test() {
    double a = 2.5;
    return a;
}
```

 It may cause information loss.

User-defined Functions

Function **return** Value

```
.....  
double cube(double x); //function prototype  
.....  
  
int main()  
{  
    .....  
    cube(3.4); // function call  
    double q = cube(1.2); // function call & Assignment  
    .....  
}  
  
double cube(double x){ // function definition  
    return x*x*x;  
}
```

The function cube
should return a
double value

1.728

CPU register

Function call and
get the return value
from the **register**

User-defined Functions

Declaring and Defining a Function

- Do not confuse between declaring and defining a function;
- **Function declaration:**
 - It **informs** the compiler of the function name, arguments, and return value types.
 - It allows the compiler to **check the variable type match**;
 - If there is no return values, just use the **void**.
- **Function definition:**
 - It supplies the actual code: function body and incurs memory allocation for this piece of code;
 - It specifies the **instructions** you want the function to do.

User-defined Functions

Declaring and Defining a Function

- Function declarations should be placed before the `main()` function. This informs the **compiler** of the function name, arguments, and return value types.

Compile

```
#include <stdio.h>
int cast_test();
int main()
{
    int a = cast_test();
    printf("%d\n", a);
    return 0;
}

int cast_test() {
    double a = 2.5;
    return a;
}
```

```
#include <stdio.h>

int main()
{
    int a = cast_test();
    printf("%d\n", a);
    return 0;
}

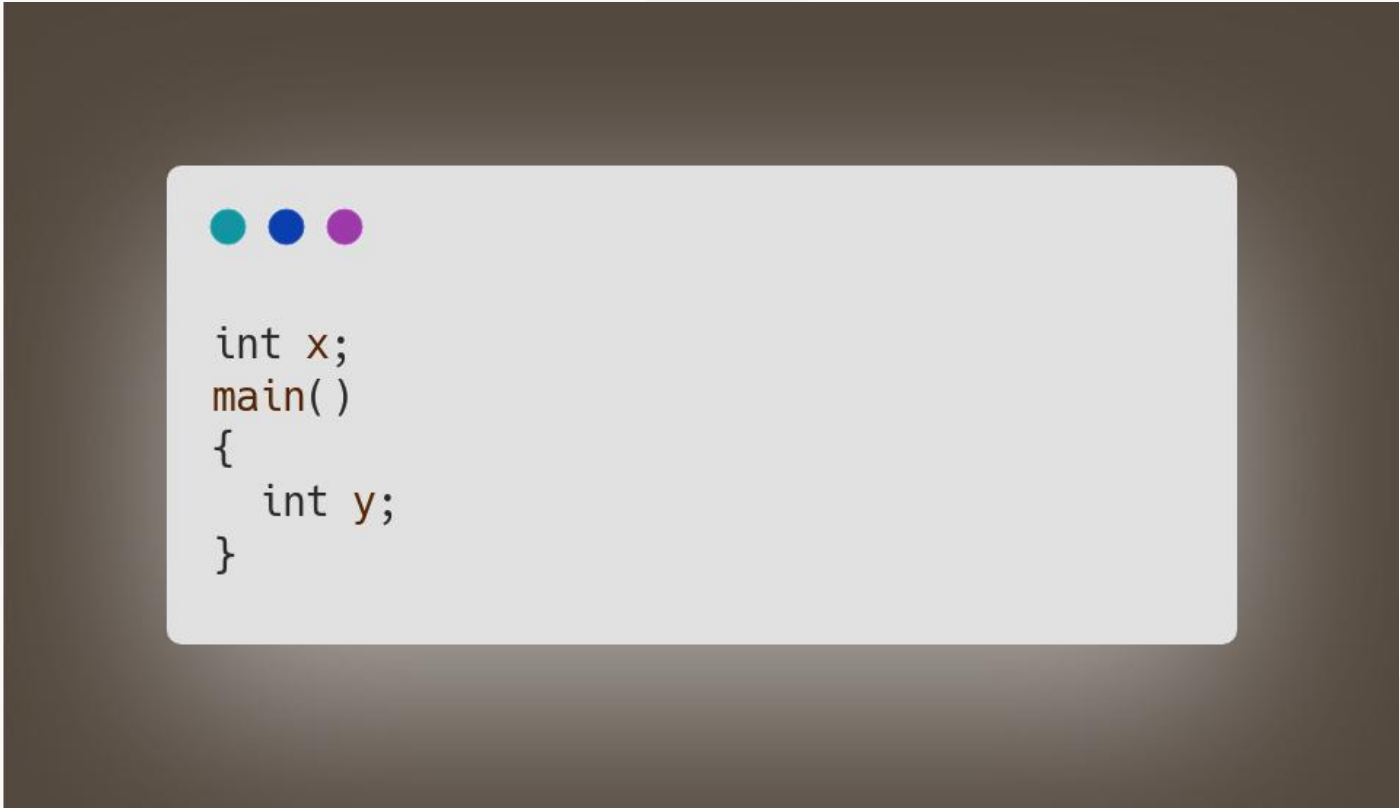
int cast_test() {
    double a = 2.5;
    return a;
}
```

Without declarations,
compiler doesn't
know how to handle
it.

Variable Scope

Variables: Global or Local?

- Variables can be declared **inside** or **outside** of a function:

A code editor window with a light gray background and a dark gray border. It contains the following C code:

```
int x;  
main()  
{  
    int y;  
}
```

The code is written in a monospaced font. The variable 'x' is declared outside the 'main' function, and 'y' is declared inside it.

- x is declared **outside** `main()`, so it is a global variable, which means that it is **visible everywhere**;
- y is declared **inside** `main()`, so it is a **local** variable, which means it is **only visible inside** `main()`.

Variable Scope

Variables: Global or Local?

- Initially it may **seem easier to use only global variables since they are available everywhere**;
 - It does not need to be passed between functions;
- However:
 - Global variables can pose **the risk of making the information incorrect**. Why is that?
 - Global variables can also lead to **inefficient memory usage**. Why is this a concern?
- So it is always a good practice to use **local variables and pass parameters**.

Variable Scope

Variables: Global or Local?

- **Global variables:** Variables declared outside of any function, they retain their values **throughout program execution** (i.e. use memory);

```
#include <stdio.h>
```

```
int x=24, y=10;  
int result;
```

```
int main()  
{  
    .....  
    return 0;  
}
```



Created at the beginning of the program.



Cleaned up at the end of whole program.

Variable Scope

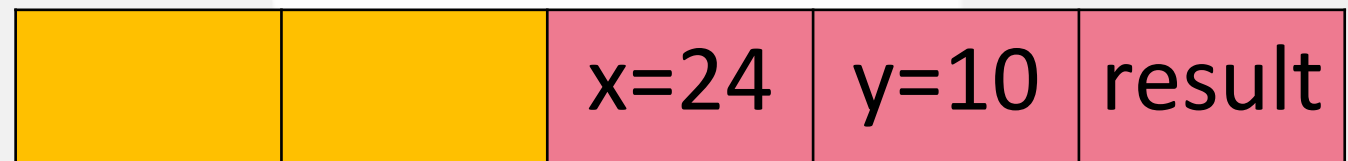
Variables: Global or Local?

- Variables declared inside a block are **local** variables and they are restricted within the corresponding **block scope**;

```
#include <stdio.h>
void test();
```

```
int main()
{
    .....
}
```

```
void test() {
    int x=24, y=10;
    int result;
    .....
}
```



Create at the beginning of function.



Cleaned up when the function terminated.

Variable Scope

Variable Scope

- The extent or range of the **visibility** of a variable within a program is often called as the **scope**;

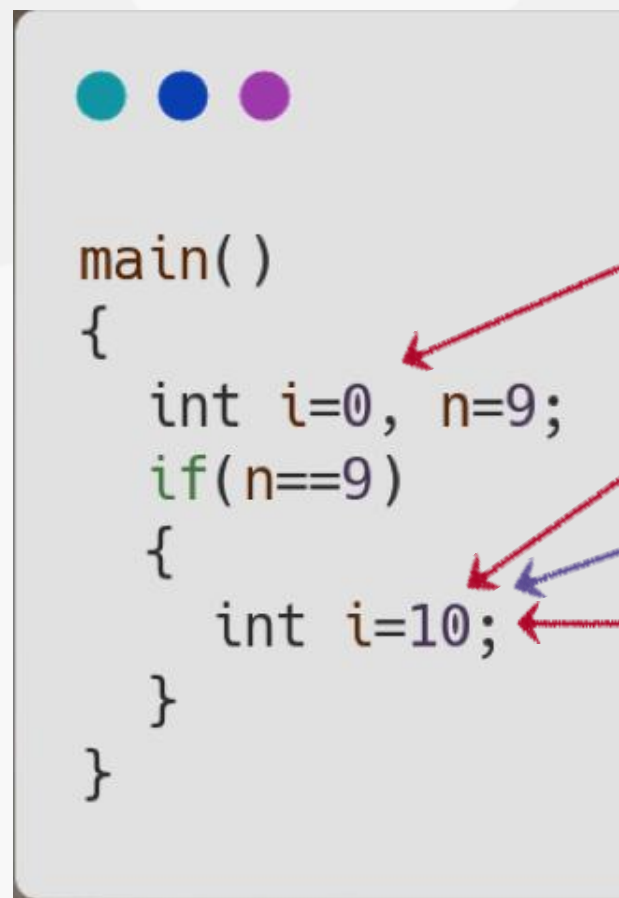
```
void function1(int x0, int y0, int w, int h) {  
    int xr = x0 + w - 1;  
}  
void function2() {  
    printf ("Left pixel=%d Right pixel=%d\n", x0, xr);  
}    /* incorrect – x0 and xr are out of scope*/
```

- Here, xr and x0 may only be used within the function function1();
 - They are **local variables** with respect to function1();
 - They are **out of the scope** with respect to the function2().

Variable Scope

Block Scope ... {}

- Variables may be declared following the **beginning of any compound statement {...}**;
- Variable local to a block hide external variables with the same name:



```
main( )  
{  
    int i=0, n=9;  
    if(n==9)  
    {  
        int i=10;  
    }  
}
```

they are NOT THE SAME
variable

inside the if block it is used

Outside the block only this i
exists

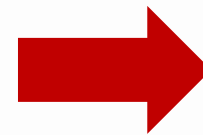
Variable Scope

Block Scope ... {}

- Example:

```
#include <stdio.h>

int main()
{
    int age = 18;
    {
        int age = 20;
        printf("My age is %d\n", age);
    }
    printf("My age is %d\n", age);
    return 0;
}
```



Output:

```
My age is 20
My age is 18
```

Variable Scope

The Scope Rule

- **Global variables**: Variables declared outside of any function, they retain their values **throughout program execution** (i.e. use memory);
- Variables declared inside a block are **local** variables and they are restricted within the corresponding **block scope**;
- Variables declared **outside ALL** the functions are in the **file scope**.
 - Variables with the file scope can be called anywhere in a file.



We will learn how to split functions across multiple files in Lecture 06.

Break

Function Calling

By Value

- In C, when functions are called with arguments, they are termed as "call by value":
 - **A copy of the data is passed to the function;**
 - The function **cannot modify the original variable value.**

Calling the Function	Function Code
<pre>int y=10, z; z=test(y); printf("y=%d",y);</pre>	<pre>int test(int y) { y = y*y; return y; }</pre>

Function Calling

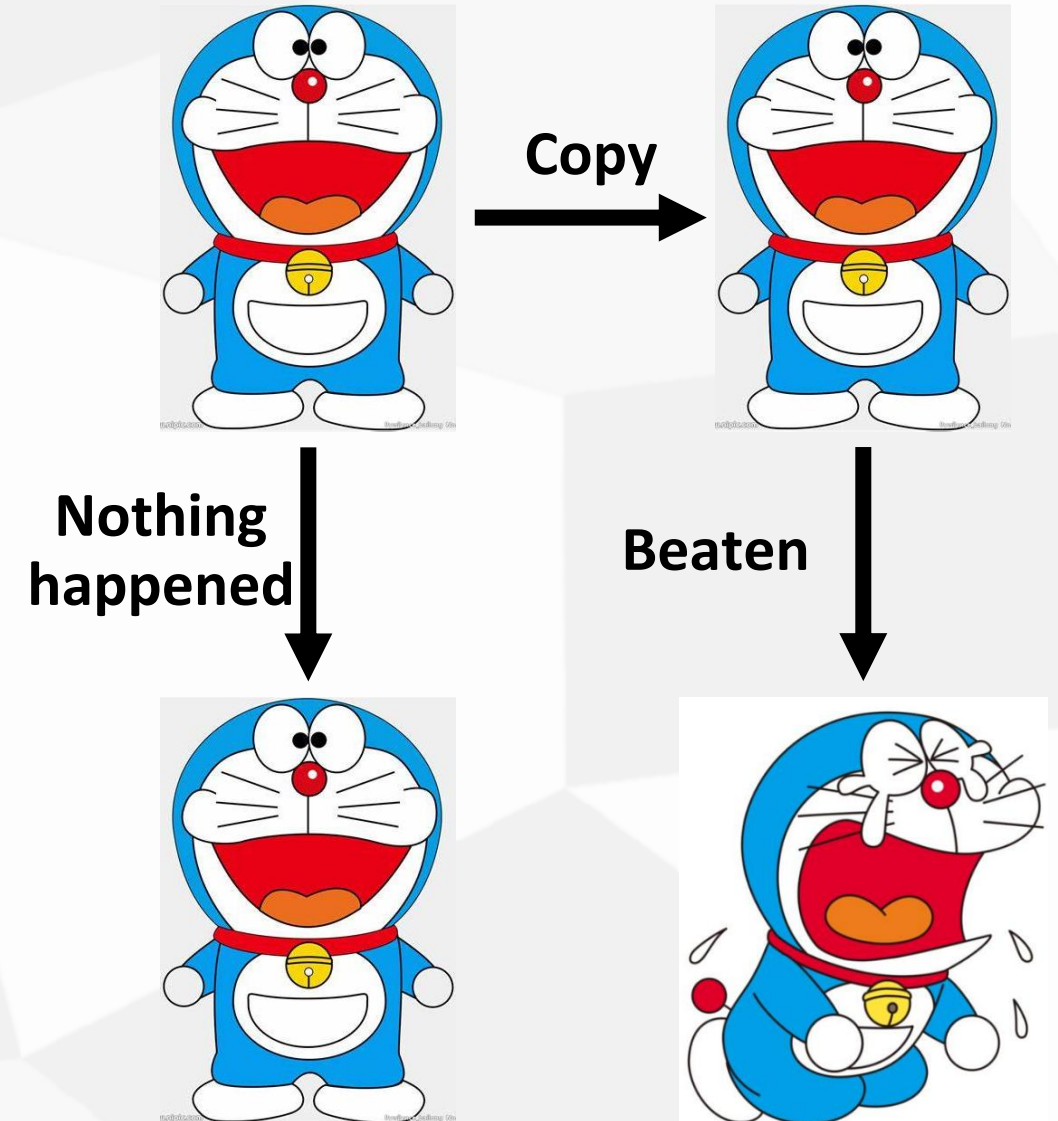
By Value

- Example

```
#include <stdio.h>

int test(int);
int main()
{
    int y=10, z;
    z = test(y);
    printf("%d\n", y);
    return 0;
}

int test(int y) {
    y = y*y;
    return y;
}
```

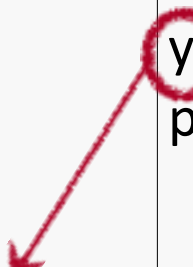


- **Q:** How can the calling function be changed to simulate **call by reference** (i.e. update the calling variable)?

Function Calling

By Value

- In C, when functions are called with arguments, they are termed as “call by value”:
 - A copy of the data is passed to the function;
 - The function cannot modify the original variable value.

Calling the Function	Function Code
<pre>int y=10, z; y=test(y); printf("y=%d",y);</pre> 	<pre>int test(int y) { y = y*y; return y; }</pre>

- **Reassign the return value to the original variable;**
- **Q:** Is it possible to change the value in the function?

Function Calling

Recursion

- A recursive function is a function that calls itself;
- A classical example is the factorial function defined:

$$n! = n \times (n - 1) \times (n - 2) \dots \times 1$$

- It can be simplified as:

$$n! = n \times (n - 1)!$$

- We can solve it by for looping, but we can also solve it by recursion function:

$$n! = n \times (n - 1)!$$

Calling ↘ ↗ Return

$$(n - 1)! = (n - 1) \times (n - 2)!$$

Calling ↘ ↗ Return

$$(n - 2)! = (n - 2) \times (n - 3)!$$

Calling ↘ ↗ Return

.....

Function Calling

Recursion

- A recursive function is a function that calls itself;
- A classical example is the factorial function defined as $0!=1$ and $n!=n*(n-1)!$



```
int factorial (int n)
{
    if (n<=1)
        return 1;
    else
        return n*factorial(n-1);
}
```

Function Calling

Recursion

```
z = factorial(3);
```

return 3*2

input 3

```
int factorial(int n)
{
    if (n<=1)
        return 1;
    else
        return n*factorial(n-1);
}
```

return 2*1

input 2

```
int factorial(int n)
{
    if (n<=1)
        return 1;
    else
        return n*factorial(n-1);
}
```

return 1

input 1

```
int factorial(int n)
{
    if (n<=1)
        return 1;
    else
        return n*factorial(n-1);
}
```

C Preprocessor

- This is conceptually a separated first step in the whole compilation process;
 - **#include** “filename” / <filename>: File inclusion:
 - “” means **teaching the file where the source program was found**;
 - <> means following an **implementation-defined rule** to find the file.
 - **#define** name replacement_text: Macro substitution
 - #define max(A, B) ((A)>(B)? (A):(B))
 - (It is to avoid the functions all overhead)

C Preprocessor

#define

- As well as using the define to make replacements in your code for constants, you can also use define to produce a function.

```
#include<stdio.h>
#define MULT(a,b) (a*b)
main(){
    int w=2,x=3,y=4,z=5;
    printf("%d",MULT((w+x),(y+z)));
}
```

macro



Avatar

$(w+x)*(y+z)$

C Preprocessor

Using the Header Files

- Complex programs are difficult to manage as a single source file;
 - They should be divided into modules;
 - **Each module is in charge of one specific functionality.**
- Function **declarations and function definitions should be put in separate files:**
 - **Function declarations** are placed in the **.h file**;
 - **Function definitions** should be placed in the **.c file**.

C Preprocessor

Examples of Header Files

Head files	Usage
math.h	contains math function prototypes
stdio.h	contain input / output function prototypes
string.h	contains string function prototypes

- The complied code for the actual function is stored in the **library** file;
- The code in a library is used during the **linking**;

C Preprocessor

Your own header files

- You can create your **own libraries** of your own functions;
- Your library files must be supplied for compiling;
- The compiler linker links the code in the libraries to the function that is called in your program;
- If you write your **own header files**, you need to provide it at the compiling time of the .c files where the functions are defined.
 - `#include<blablabla.h>`

Random Number Generation

- Random numbers are used in many programs and simulations, e.g., simulating noise, playing games, and predictions, etc.;
- The random number generator in C is **rand()**. It generates an integer between **0** and **RAND_MAX** (a constant defined in **<stdlib.h>**):

`i = rand();`

- The function **srand()** uses a seed (e.g., the CPU clock) as the **initial state** for a **pseudo random** sequence.

Random Number Generation

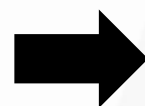
```
#include <stdio.h>

int main()
{
    srand(100); //Seed for rand()
    for(int i=0;i<8;i++) {
        printf("%d\n", 1+(rand()%6));
    }
    return 0;
}
```



- However, the code will generate the same sequence every time.

8 different numbers?



```
#include <time.h>
srand(time(NULL));
```

Random Number Generation

- Changing the **seed** value will change the sequence;
- **The same seed will always produce the same apparently random sequence**; —> when we want to use this?
- Try to use:

```
#include<time.h>
long t;
t=time(0); /* it returns the current system clock value */
srand(t);
```

Random Number Generation

```
#include <stdio.h>
#include <time.h>

int main()
{
    long t;
    t = time(0); //Current System Clock time.
    srand(t); //Seed for rand()

    for(int i=0; i<8; i++) {
        printf("%d\n", 1+(rand()%6));
    }
    return 0;
}
```



Function Structure (Suggestion)

- **Pre-Processors:** Anything with a #;
- **Function Prototypes:** Declare your functions;
- **Global Variables:** Be careful of using these;
- **Function main():** The beginning of the program;
- **Declare ALL the variables first:**
 - Beginning of any of the compound statements;
 - Try not to declare new variables in the middle of your code;
 - (C99 standard allows this, however, C90 does not allow this)
- **Coding and function calls:** Write your own codes;
- **End the function main();**
- **Define your own functions:** The same format as the function main().

Thanks